

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: May 14, 2026

Lab # 12

Implementation of DSA using OpenSSL

Environment Preparation: The mkdir command creates a dedicated directory named Lab_12, and the openssl version confirms that OpenSSL 3.5.5 is installed. The cd command is then used to move into the newly created workspace.

```
(kali㉿kali)-[~/Documents]
$ mkdir Lab_12

(kali㉿kali)-[~/Documents]
$ openssl version
OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)

(kali㉿kali)-[~/Documents]
$ cd Lab_12
```

Part B: DSA Implementation

1. Generate DSA Parameters

The openssl dsaparam command generates the necessary global parameters for the DSA algorithm with a 2048-bit key size. The ls -la command confirms the creation of the dsa_param.pem file required for the next steps.

```
(kali㉿kali)-[~/Documents/Lab_12]
$ openssl dsaparam -out dsa_param.pem 2048

(kali㉿kali)-[~/Documents/Lab_12]
$ ls -la dsa_param.pem
-rw-rw-r-- 1 kali kali 816 May  7 13:55 dsa_param.pem
```

2. Generate DSA Private Key

The openssl gendsa command uses the previously generated parameters to create a unique DSA private key. Checking the directory with ls verifies that dsa_private.pem has been successfully created with restricted permissions.

```
(kali㉿kali)-[~/Documents/Lab_12]
$ openssl gendsa -out dsa_private.pem dsa_param.pem

(kali㉿kali)-[~/Documents/Lab_12]
$ ls -la dsa_private.pem
-rw----- 1 kali kali 879 May  7 13:56 dsa_private.pem
```

3. Extract DSA Public Key

The openssl dsa utility reads the private key file to derive and output the corresponding public key as dsa_public.pem. This public key will be used by third parties to verify the authenticity of signed messages.

```
(kali㉿kali)-[~/Documents/Lab_12]
$ openssl dsa -pubout -in dsa_private.pem -out dsa_public.pem
read DSA key
writing DSA key

(kali㉿kali)-[~/Documents/Lab_12]
$ ls -la dsa_public.pem
-rw-rw-r-- 1 kali kali 1190 May  7 13:57 dsa_public.pem
```

4. Create Digital Signature

The openssl dgst command hashes the message using SHA-256 and signs the hash with the DSA private key to produce dsa_signature.bin. The xxd hex dump shows the raw binary signature data associated with the message.

```
(kali@kali)-[~/Documents/Lab_12]
$ openssl dgst -sha256 -sign dsa_private.pem -out dsa_signature.bin secret_msg.txt

(kali@kali)-[~/Documents/Lab_12]
$ ls -la dsa_signature.bin
-rw-rw-r-- 1 kali kali 62 May  7 13:59 dsa_signature.bin

(kali@kali)-[~/Documents/Lab_12]
$ xxd dsa_signature.bin
00000000: 303c 021c 11d1 aa11 90ae c25d e229 f571 0<.....].).q
00000010: f45c e755 4a83 bcfc 5000 5c77 871b f61d .\UJ...P.\w....
00000020: 021c 609a 9b44 6ff8 883f b3b4 9ad4 1af9 ..`..Do..?.....
00000030: 2a01 65bc 9582 638d 67fc 8e47 05b2 *.e...c.g..G..
```

5. Verify Digital Signature

The openssl dgst -verify command uses the DSA public key to validate the signature against the original message file. The "Verified OK" result confirms that the valid private key signed the message and has not been tampered with.

```
(kali@kali)-[~/Documents/Lab_12]
$ openssl dgst -sha256 -verify dsa_public.pem -signature dsa_signature.bin secret_msg.txt
Verified OK
```

Observation

While we successfully generated keys and created signatures using DSA, we encountered a specific OpenSSL error when trying to use the DSA public key for encryption. The error message stated: "Public key operation not supported by this key type." This highlights that DSA is strictly a signature algorithm and isn't intended for purposes of confidentiality. Moreover, we noted that RSA keys, set at 2048 bits, yielded larger ciphertexts and signatures compared to those produced by DSA.

```
(kali@kali)-[~/Documents/Lab_12]
$ openssl pkeyutl -encrypt -in secret_msg.txt -out dsa_encrypted.bin -pubin -inkey dsa_public.pem
pkeyutl: Error initializing context
40B74B20F37F0000:error:03000096:digital envelope routines:evp_pkey_asym_cipher_init:operation not supported for this keytype:../crypto/evp/asmcipher.c:202:

(kali@kali)-[~/Documents/Lab_12]
$ openssl rsautl -encrypt -in secret_msg.txt -out dsa_encrypted.bin -pubin -inkey dsa_public.pem
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
RSA operation error
40178AB3D47F0000:error:03000096:digital envelope routines:evp_pkey_asym_cipher_init:operation not supported for this keytype:../crypto/evp/asmcipher.c:202:
```

The pkeyutl command fails because the **Digital Signature Algorithm (DSA)** is mathematically restricted to signing; unlike RSA, it does not support encryption. This error confirms that DSA keys are specialized exclusively for authenticity and integrity, not for confidentiality.

Furthermore, attempting to use the deprecated rsautl utility results in a failure because it cannot perform RSA-specific operations on a DSA key type. This illustrates that modern OpenSSL versions require the pkeyutl tool and strict adherence to the specific functions supported by each cryptographic algorithm.

Result

- **Parameter and Key Generation:** The successful generation of 2048-bit global parameters produced the file `dsa_param.pem`. These parameters were then used to generate a DSA private key (`dsa_private.pem`) and extract the corresponding public key (`dsa_public.pem`).
- **Signing Process:** Using the private key, we signed the file `secret_msg.txt` with SHA-256 hashing, which produced a binary signature file named `dsa_signature.bin`. A hex dump of this signature reveals the encrypted hash, serving as the digital fingerprint for the message.

- **Verification Status:** The system performed a verification process using the public key to check the original file against its signature. The terminal output confirmed "**Verified OK**," indicating that the signature is valid.

Conclusion

This implementation showcases the fundamentals of the Digital Signature Algorithm (DSA). The "Verified OK" message highlights how asymmetric cryptography can ensure both authenticity—verifying the sender's identity—and integrity, confirming that the message remained unchanged during transmission. Unlike RSA, which is frequently employed for both encryption and signing, this DSA workflow underscores the algorithm's specific functions in non-repudiation and verification. The match between the signature generated by the private key and the output verified by the public key confirms the integrity of the entire cryptographic process.

- End of Lab Task -